

PSYC 51.17: Models of Language and Communication

Lecture 15: Attention Mechanisms

Week 5, Lecture 1 - From Seq2Seq to Attention

PSYC 51.17: Models of Language and Communication

Winter 2026

Winter 2026

1. **The Context Problem:** Why static embeddings aren't enough
2. **Sequence-to-Sequence Models:** The foundation
3. **The Bottleneck Problem:** Why vanilla Seq2Seq struggles
4. **Attention Mechanisms:** The breakthrough innovation
5. **How Attention Works:** Step-by-step computation
6. **Visualizing Attention:** Interpreting the weights

*Goal: Understand why and how attention
revolutionized NLP*

The Context Problem

Why do we need context-aware models?

Example: The word "bank"

1. "I deposited money at the **bank**" (financial institution)
2. "We sat by the river **bank**" (riverside)
3. "The plane started to **bank** left" (tilt/turn)

Static Embeddings (Word2Vec):

- One vector per word

Context-Aware Models:

- Different representation per occurrence

Sequence-to-Sequence Models

The breakthrough for variable-length input/output problems

Applications:

- Machine Translation: English → French
- Summarization: Long text → Short summary
- Question Answering: Question + Context → Answer
- Dialogue Systems: User input → System response

Concrete Example: Machine Translation

Input: "The cat sat on the mat" (6 tokens)

Encoder-Decoder Architecture

Two-part architecture: Encode then Decode

Encoder:

- Reads input sequence
- Compresses to fixed-size vector
- Captures semantic meaning

Worked Example:

1	Input: "The"	
	→ "cat" →	
	"sat"	
2	↓ ↓ ↓	5
3	$h_1 \rightarrow h_2 \rightarrow$	
	$h_n = \text{context}$	

The Seq2Seq Bottleneck Problem

Challenge: All information compressed into single vector!

The Problem

- Long sequences → information loss
- Fixed-size context vector is a bottleneck
- Early tokens forgotten by the time we reach the end
- Performance degrades with sequence length

Concrete Example: Long Sentence Translation

Input (20 words): "The quick brown fox jumps over the lazy dog while the cat watches from the warm sunny windowsill nearby"

Problem: All 20 words must fit into one 256-dim vector!

Attention Mechanism: The Big Idea

Instead of compressing everything into one vector...

Let the decoder look at all encoder hidden states!

1 | $h_1 \rightarrow h_2 \rightarrow h_3 \rightarrow h_4 \rightarrow \text{Encoder} \rightarrow s_t \rightarrow \text{Decoder} \rightarrow$

Key Insight: When generating "chat" (cat), pay more attention to "cat" in the input!

Reference: Bahdanau et al. (2015) - "Neural Machine Translation by Jointly Learning to Align and Translate"

Winter 2026

2. **Normalize:** Convert scores to probabilities (softmax)

- $\alpha_{\{t,i\}} = \exp(e_{\{t,i\}}) / \sum_j \exp(e_{\{t,j\}})$

3. **Context:** Weighted sum of encoder states

- $c_t = \sum_i \alpha_{\{t,i\}} * h_i$

Step 2: When generating "chats", compute scores

- Decoder state $s = [0.5, 0.6]$
- Score with h_1 : $s \cdot h_1 = 0.5 \times 0.2 + 0.6 \times 0.8 = \mathbf{0.58}$
- Score with h_2 : $s \cdot h_2 = 0.5 \times 0.9 + 0.6 \times 0.3 = \mathbf{0.63}$
- Score with h_3 : $s \cdot h_3 = 0.5 \times 0.4 + 0.6 \times 0.7 = \mathbf{0.62}$

Step 3: Softmax to get attention weights

- $\alpha = \text{softmax}([0.58, 0.63, 0.62]) = [\mathbf{0.31}, \mathbf{0.35}, \mathbf{0.34}]$

Step 4: Context = weighted sum

Attention Score Functions

Different ways to compute the score

1. Additive (Bahdanau):

```
1 # score =  
  v^T *  
  tanh(W1*s +  
    W2*h)  
2 score = v @  
  tanh(W1 @ s
```

3. Dot-Product:

```
1 # score =  
  s^T * h  
2 score = s @  
  h
```

10

attention weights

Input: "The European Economic Area"

Output: "La zone économique européenne"

1 | The European Economic Area

Observations:

- Diagonal pattern for similar word order
- Model learns alignment automatically!
- "européenne" attends to "European" (reordering handled!)
- No need for explicit word alignment annotations

Attention provides interpretability: we can see what

- Better BLEU scores on translation tasks
- Handles long-range dependencies
- More robust to sequence length

3. Provides Interpretability

- Can visualize what the model focuses on
- Helps debug and understand model behavior

Real-World Impact of Attention

Attention mechanisms revolutionized multiple domains:

Machine Translation:

- Google Translate (2016)
- DeepL
- Facebook translations
- Dramatic quality improvements

Text Summarization:

- News article summarization

Question Answering:

- Reading comprehension
- Search engines
- Virtual assistants

Speech Recognition:

- Attend to acoustic features
- Better transcription accuracy

Implementing Attention in PyTorch

Simple attention mechanism implementation

```
1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4
5 class BahdanauAttention(nn.Module):
6     def __init__(self, hidden_dim):
7         super().__init__()
8         self.W_dec = nn.Linear(hidden_dim, hidden_dim)
9         self.W_enc = nn.Linear(hidden_dim, hidden_dim)
10        self.v = nn.Linear(hidden_dim, 1)
11
12    def forward(self, decoder_hidden, encoder_outputs):
13        # Compute scores: how relevant is each encoder state?
14        dec = self.W_dec(decoder_hidden).unsqueeze(1) # [batch, 1, hidden]
15        enc = self.W_enc(encoder_outputs) # [batch, seq_len, hidden]
```

14

Winter 2026

continued...

Implementing Attention in PyTorch

```
16 scores = self.v(torch.tanh(dec + enc)) # [batch, seq_len, 1]
17
18 # Normalize to get attention weights (sum to 1)
19 attn_weights = F.softmax(scores, dim=1) # [batch, seq_len, 1]
20
21 # Weighted sum of encoder outputs
22 context = torch.sum(attn_weights * encoder_outputs, dim=1)
23
24 return context, attn_weights.squeeze(-1)
```

Winter 2026

Using the Attention Module

Complete example with sample data

```
1 # Initialize attention module
2 attn = BahdanauAttention(hidden_dim=64)
3
4 # Sample encoder outputs (3 words, 64-dim hidden state)
5 encoder_outputs = torch.randn(1, 3, 64) # [batch=1, seq_len=3, hidden=64]
6
7 # Current decoder hidden state
8 decoder_hidden = torch.randn(1, 64) # [batch=1, hidden=64]
9
10 # Compute attention
11 context, weights = attn(decoder_hidden, encoder_outputs)
12
13 print(f"Context shape: {context.shape}") # [1, 64]
14 print(f"Attention weights: {weights}") # [1, 3] - sums to 1.0!
15
16 # Example output:
17 # Attention weights: tensor([[0.28, 0.45, 0.27]])
18 # "The" "cat" "sat"
```

16

Winter 2026

continued...

Using the Attention Module

19 | # Model focuses most on "cat" when generating next output word!

Winter 2026

- What is the time complexity of attention?
- How does it scale with sequence length?
- When might this be a problem?

3. Interpretability:

- Can we always trust attention weights as explanations? 18
- What about when attention is uniform across all inputs?

- The context problem in NLP
- Sequence-to-sequence architecture
- The bottleneck problem
- How attention mechanisms work
- Attention as alignment and interpretation

Next lecture (Lecture 13):

- : Attention within a sequence
- : "Attention is All You Need"
- : The new framework

- Fixed-size context vector limits performance
- Information loss for long sequences

3. **Attention is the Solution**

- Dynamic access to all encoder states
- Weighted combination based on relevance
- Attention weights sum to 1.0 (probability distribution)

- Introduced additive attention mechanism
- Solved the bottleneck problem

\item **Luong et al. (2015)** - "Effective Approaches to Attention-based Neural Machine Translation"

- Multiplicative attention variants
- Global vs. local attention

\item **Vaswani et al. (2017)** - "Attention Is All You Need"

Questions?

Discussion Time

Office Hours Topics:

- Implementing attention from scratch
- Different attention mechanisms
- Debugging attention-based models
- Assignment 4 preparation

Thank you!

See you next lecture for Transformers!

Winter 2026